

Nombre y Apellido: N° Legajo:

Segundo Parcial de Programación Imperativa

08/06/2026

	Ejercicio 1	Ejercicio 2	Ejercicio 3	Nota
Calificación	/4	/4	/2	

- ★ **Condición mínima de aprobación: Sumar 5 (cinco) puntos.**
- ★ **Los ejercicios que no se ajusten estrictamente al enunciado, no serán aceptados.**
- ★ **No usar variables globales ni static.**
- ★ **No es necesario escribir los #include**
- ★ **Escribir en esta hoja Nombre, Apellido y Legajo**
- ★ **Antes de retirarse del aula confirmar con el docente que se haya recibido la entrega**

Ejercicio 1

Se desea implementar un TAD para administrar las distintas **sucursales de una cadena de librerías**.

Cada sucursal se identifica con un número entero positivo múltiplo de 10 (10, 20, 130, etc.). La cantidad máxima de sucursales no se conoce de antemano, pero sí se sabe que no habrá muchos "huecos" en la numeración, por ejemplo pueden existir las sucursales 10, 30, 40, 60, 70, 90, etc. pero no únicamente las sucursales 10, 1200 y 8900

En cada sucursal se registran los libros disponibles. Para cada libro se registra un título (un string corto) que es único, y la cantidad de ejemplares disponibles.

En una misma sucursal no puede haber dos libros con el mismo título.

Un mismo libro puede estar disponible en distintas sucursales.

Para ello se definió la siguiente interfaz:

```
typedef struct libraryCDT * libraryADT;

struct bookData {
    char * title;
    size_t stock;
};

libraryADT newLibrary(void);

void freeLibrary(libraryADT lib); // NO IMPLEMENTAR

/**
 * Agrega -si no estaba- un nuevo libro a una sucursal.
 * Si ya existe ese título, incrementa el stock actual
 * El parámetro stock indica cuántos ejemplares se agregan de ese libro.
 * Si la sucursal es inválida retorna cero, si no retorna el stock actual
 * de ese libro
 */
size_t addBook(libraryADT lib, size_t branch, const char * title, unsigned int stock);
```

```

/**
 * Retorna la cantidad de libros distintos que hay en una sucursal.
 * Si la sucursal es inválida retorna cero.
 */
size_t books(const libraryADT lib, size_t branch);

/**
 * Retorna la cantidad total de ejemplares disponibles en una sucursal.
 * Si la sucursal es inválida retorna cero.
 */
size_t stock(const libraryADT lib, size_t branch);

// Funciones para iterar por sucursal, alfabéticamente por título
// Permite iterar en forma paralela por varias sucursales (ver test)
void toBeginByBranch(libraryADT lib, size_t branch);

/* 1 si hay siguiente libro para una sucursal, 0 si no */
int hasNext(const libraryADT lib, size_t branch); // NO IMPLEMENTAR

struct bookData next(libraryADT lib, size_t branch); // NO IMPLEMENTAR

```

Se pide **implementar todos los structs y ÚNICAMENTE las siguientes funciones,**

- **newLibrary**
- **addBook**
- **books**
- **stock**
- **toBeginByBranch**

junto con todas las funciones que sean invocadas por las mismas.

Programa de prueba:

```

int main(void) {
    libraryADT lb = newLibrary();
    // Sucursal 15 es inválida
    assert(addBook(lb, 15, "Moby Dick english edition", 8)==0);

    assert(addBook(lb, 20, "Moby Dick english edition", 8)==8);
    assert(addBook(lb, 20, "Moby Dick english edition", 7)==15);

    assert(addBook(lb, 30, "Moby Dick english edition", 18)==18);

    assert(addBook(lb, 20, "Moby Dick ilustrado", 8)==8);

    char aux[100];
    strcpy(aux, "Ficciones, Borges");
    assert(addBook(lb, 20, aux, 10)==10);
    strcpy(aux, "1984");
    assert(addBook(lb, 20, aux, 1)==1);
    strcpy(aux, "MOBY DICK english edition");
    assert(addBook(lb, 20, aux, 10)==25); // Había 15, se suman 10

    assert(books(lb, 20)==4);
    assert(stock(lb, 20)==44);

    assert(books(lb, 30)==1);
    assert(stock(lb, 30)==18);

    assert(books(lb, 40)==0);
    assert(stock(lb, 40)==0);
}

```

```

toBeginByBranch(lb, 20);
struct bookData book;
book = next(lb, 20);
assert(book.stock == 1);
assert(strcmp(book.title, "1984")==0);
free(book.title);
book = next(lb, 20);
assert(book.stock == 10);
assert(strcmp(book.title, "Ficciones, Borges")==0);
free(book.title);

toBeginByBranch(lb, 30);

book = next(lb, 20);
assert(book.stock == 25);
assert(strcmp(book.title, "Moby Dick english edition")==0);
free(book.title);
book = next(lb, 20);
assert(book.stock == 8);
assert(strcmp(book.title, "Moby Dick ilustrado")==0);
free(book.title);

assert(hasNext(lb, 20)==0);
assert(hasNext(lb, 30)==1);

freeLibrary(lb);
puts("OK");
return 0;
}

```

Ejercicio 2

Se implementará un TAD para generar **números enteros positivos al azar**, y obtener algunas estadísticas sobre los mismos.

Para ello se definió la siguiente interfaz:

```

typedef struct randomCDT * randomADT;

/**
 * TAD para almacenar números aleatorios entre 1 y maxValue inclusive
 * Si maxValue es 0 retorna NULL
 */
randomADT newRandomADT(unsigned char maxValue);

/**
 * Crea y almacena qty números aleatorios menores o iguales a maxValue
 */
void newRandoms(randomADT adt, size_t qty);

/**
 * Crea y almacena un número aleatorio menor o igual a maxValue
 */
void newRandom(randomADT adt);

/**
 * Retorna cuántos números distintos se han almacenado
 */
unsigned char numbers(const randomADT adt);

```

```

/**
 * Retorna el número que más veces apareció (si hay más de uno cualquiera de ellos)
 * Si no se generó ningún número retorna -1
 */
int mostFrequent(const randomADT adt);

void freeRandomADT(randomADT adt);

```

Se pide **implementar el TAD COMPLETO**.

Se puede invocar la función `randInt` vista en clase (no es necesario implementarla).

Ejercicio 3

Dada la siguiente definición de estructuras para una **lista lineal**:

```

typedef struct node {
    int head;
    struct node * tail;
} node;

typedef node * TList;

```

Implementar la función recursiva `deleteAll` que reciba

- **list:** una lista (puede estar vacía)
- **value:** un número entero
- **count:** un puntero a int no inicializado

La función debe **eliminar de list** todos los elementos que coincidan con **value**, y dejar en ***count** cuántos elementos se eliminaron.

Ejemplos:

- Si la lista fuera 2 -> 5 -> 1 -> 5 -> 4 -> 5 -> 7 y value es 5, la lista queda 2 -> 1 -> 4 -> 7 y en *count deja el valor 3
- Si la lista fuera 2 -> 85 -> 1 -> 15 -> 4 -> 8 -> 7 y value es 5, la lista queda igual y en *count deja el valor 0
- Si la lista es vacía deja en *count el valor 0
- Si la lista fuera 5 -> 2 -> 1 -> 5 -> 5 -> 4 -> 8 -> 7 -> 5 y value es 5, la lista queda 2 -> 1 -> 4 -> 8 -> 7 y en *count deja el valor 4
- Si la lista fuera 5 -> 5 -> 5 y value es 5, la lista queda vacía y en *count deja el valor 3

NO SE ADMITIRÁ UNA SOLUCIÓN QUE TENGA UN CICLO DENTRO DE LA FUNCIÓN.

NO DEFINIR MACROS NI FUNCIONES AUXILIARES.